

Phase 2 Detailed Task Distribution

As we step into Phase 2 (Automated RLAIF Data Engine, NLI Calibration, and Edge Generation), here is our exact division of labor:

M1: Khushi (Lead Architect & Core Implementation)

- **Three-Way Decision Policy Integration:** Upgrade the cross-encoder verification gate from a binary pass/fail check into the full three-way deterministic state machine routing: **ANSWER** ($P(\text{Entailment}) \geq 0.75$), **CLARIFY** ($P(\text{Neutral}) > 0.40$), and **ABSTAIN** ($P(\text{Contradiction}) > 0.30$).
- **Empirical Ablation Framework:** Write the evaluation execution script to run systematic threshold sweeps across our benchmark queries to map precision, recall, and selective risk curves.
- **Module Integration & Orchestration:** Merge Praisys's RLAIF dataset generation script and Meghana's GBNF generation module into the central master repository.
- **Paper Drafting:** Begin writing core sections of the research manuscript (*Introduction, Methodology, and Experimental Setup*).

M2: Meghana (Development Support: SLM Generation, GBNF & UI)

- **GBNF Grammar Schema Construction:** Create the constraints.gbnf context-free grammar file to enforce strict JSON structure compliance on the Qwen-2.5-1.5B model outputs (including fields for evidence_chunk_id, parameter_identified, and recommended_action_telugu).
- **Local SLM Engine Integration:** Wire up llama.cpp Python bindings with the quantized Qwen-2.5-1.5B-Instruct model using logit-masking.
- **Streamlit Edge Interface:** Build the local browser UI (localhost:8501) featuring real-time diagnostic telemetry panels displaying retrieval scores, decision states, and JSON citation outputs.
- **Integration Testing:** Work with M1 to ensure end-to-end Time-to-First-Token (TTFT) stays under 3.0 seconds on CPU.

M3: Jyothish (Documentation, Logbook & Diagrams)

- **Engineering Logbook:** Maintain a detailed, dated record of all weekly decisions, hyperparameter configurations, challenges, and empirical solutions.
- **Zotero & Reference Management:** Curate and organize all literature survey references (ICAR-CIBA manuals, BGE-M3, DeBERTa, DPO, and RLAIF papers) for the bibliography.
- **Publication-Ready Diagrams:** Produce high-resolution vector diagrams (using Draw.io or LaTeX TikZ) for:
 1. The Two-Stage Cascaded Retrieval Pipeline.
 2. The LangGraph Three-Way Decision State Machine.
 3. The Automated RLAIF Data Engine Architecture.
- **Repository Documentation:** Keep the GitHub README.md and setup instructions fully up-to-date.

M4: Praisys (Review Support, Data Engine & Visualizations)

- **Automated RLAIF Data Engine Script (src/generate_rlaif_data.py):** Implement the script that synthesizes cross-lingual NLI training triplets (Entailment, Neutral, Contradiction) from processed English text chunks.
- **Biological Invariant Verification Guardrails:** Build the Python assertion suite (verify_aquaculture_invariants) to filter out biologically impossible values (e.g., pH bounds outside [6.0, 9.5] or invalid dissolved oxygen levels) before dataset admission.

- **Result Visualizations & Graphs:** Generate high-resolution figures for the paper and presentations, including safety-coverage curves and latency breakdown charts.
- **Presentation, Demo & Viva Prep:** Coordinate slide decks, structure the demo evaluation flow, and compile anticipated viva questions and answers.

Phase 2 Execution Flow & Trigger Milestones

Step 1: Praisyl (The Starting Block)

- **When to start:** Immediately. Praisyl's work has zero upstream dependencies because it builds directly off the clean `chunks.json` we just finalized in Phase 1.
- **What they do:** Run `src/generate_rlaif_data.py` to ingest the chunks, synthesize the Telugu query pairs, and apply the biological invariant filter (`verify_aquaculture_invariants`).
- **The Hand-off Trigger:** Once Praisyl generates and pushes the verified training/evaluation dataset to the repository (e.g., `data/evaluation/rlaif_dataset.json`), **Khushi and Meghana are instantly unblocked to start Steps 2 and 3.**

Step 2: Khushi & Meghana (Parallel Core Development)

- **When to start:** The exact moment Praisyl's dataset file lands in the repository.
- **What Khushi does:** Takes the evaluation data and upgrades the cross-encoder verification gate into the full **Three-Way Decision Policy** (`Answer`, `Clarify`, `Abstain`) with probability thresholds.
- **What Meghana does:** Takes the output requirements and writes the `constraints.gbnf` grammar file, wires up `llama.cpp` with Qwen-2.5-1.5B, and builds the local Streamlit UI (`localhost:8501`).
- **The Hand-off Trigger:** Once Khushi locks down the routing logic and Meghana finishes the edge generation UI, the complete end-to-end pipeline is functional.

Step 3: Jyothish & Praisyl (Integration, Validation & Documentation)

- **When to start:** Runs concurrently as Step 2 wraps up, ramping up fully once the integrated pipeline is running.
- **What Jyothish does:** Documents hyperparameter logs, organizes Zotero citations, and produces the vector architecture diagrams (LangGraph State Machine, Cascaded Pipeline) as code components stabilize.
- **What Praisyl does:** Takes the final system outputs to run the ablation sweeps, generate safety-coverage curves, and compile the final presentation/viva materials.